Laporan Milestone 1 Lexical Analysis

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

Disusun oleh:

BST - brawlstars

Kevin Wirya Valerian	13524019
Ray Owen Martin	13524033
Emilio Justin	13524043
Philipp Hamara	13524101

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

Daftar Isi 2
Daftar Gambar 3
Daftar Tabel 4
I. Teori Singkat 5
A. Tabel Simbol 5
B. Analisis Leksikal 5
C. Deterministic Finite Automata 6
II. Perancangan dan Implementasi 7
A. Rancangan DFA 7
B. Implementasi Program 11
III. Pengujian 13
A. Pengujian 1 13
B. Pengujian 2 14
C. Pengujian 3 18
D. Pengujian 4 19
E. Pengujian 5 20
IV. Kesimpulan dan Saran 25
A. Kesimpulan 25
B. Saran 25
Referensi 26
Lampiran 27

Daftar Gambar

Gambar 1. Diagram DFA program lexical analyzer bahasa pemrograman Arion.....11
Gambar 3.1 Hasil Pengujian 1.....13
Gambar 3.2 Hasil Pengujian 2.....17
Gambar 3.3 Hasil Pengujian 3.....18
Gambar 3.4 Hasil Pengujian 4.....19
Gambar 3.5 Hasil Pengujian 5.....24

Daftar Tabel

Tabel 1. Definisi formal DFA program lexical analyzer bahasa pemrograman Arion..... 7

I. Teori Singkat

A. Tabel Simbol

Tabel simbol (symbol tables) adalah struktur data yang digunakan *compiler* untuk menyimpan informasi terkait berbagai konstruksi program sumber. Informasi dalam tabel ini dikumpulkan secara bertahap selama fase analisis kemudian dimanfaatkan pada fase sintesis untuk menghasilkan *target code*. Setiap entri di dalam tabel simbol mencatat detail spesifik dari sebuah *identifier*, seperti bentuk *lexeme*, tipe data, alokasi posisi dalam memori penyimpanan, serta atribut relevan lainnya.

B. Analisis Leksikal

Analisis leksikal (*lexical analysis*) adalah tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Tugas utamanya adalah untuk membaca rangkaian karakter dari program sumber, mengelompokkan mereka menjadi *lexemes,* dan menghasilkan urutan *token* untuk masing-masing *lexeme* sebagai output. Proses ini juga membuang bagian yang tidak diperlukan seperti spasi, tab, dan komentar, serta dapat mendeteksi kesalahan seperti simbol yang tidak dikenal sejak awal. Urutan token-token tersebut dikirimkan ke *parser* untuk proses analisis sintaks (syntax analysis). Tahap ini berinteraksi dengan tabel simbol. Analisis leksikal akan memasukkan sebuah entri baru ke tabel simbol ketika menemukan sebuah *lexeme* yang merupakan sebuah *identifier*.

Dalam konteks analisis leksikal, terdapat tiga terminologi utama yang saling berkaitan tetapi memiliki definisi spesifik, yaitu *token, pattern,* dan *lexeme. Token* merupakan entitas berpasangan yang terdiri dari nama token dan nilai atribut opsional. *Pattern* adalah deskripsi atau aturan formal yang mendefinisikan bentuk struktur yang dapat diambil oleh *lexeme* dari suatu *token*. *Lexeme* adalah sekuens karakter aktual di dalam program sumber yang secara spesifik memenuhi kriteria pola dari suatu *token*, dan secara resmi diidentifikasi oleh penganalisis leksikal sebagai instansi dari token tersebut.

Proses analisis leksikal dilakukan oleh komponen yang disebut *lexer*. *Lexer* membaca kode sumber dari kiri ke kanan dan menggunakan *pattern* tertentu untuk mengenali jenis token yang berbeda-beda. Ketika menemukan urutan karakter yang cocok dengan salah satu *pattern* tersebut, *lexer* membuat sebuah *token* yang terdiri dari jenis *token* dan nilai *token*.

Agar proses pembacaan tersebut dapat dijalankan secara otomatis oleh komputer, mereka harus diimplementasikan ke dalam bentuk *Finite Automata*, khususnya *Deterministic Finite Automata* (DFA).

C. Deterministic Finite Automata

Finite Automata merupakan model komputasi yang melibatkan serangkaian *states* dan proses transisi di antara *state-state* tersebut sebagai respons terhadap suatu masukan. Konsep ini memiliki peranan krusial dan diaplikasikan secara luas dalam pengembangan berbagai jenis perangkat lunak. Salah satu penerapan utamanya adalah pada perancangan komponen penganalisis leksikal di dalam sebuah *compiler.*

Istilah 'deterministik' dalam *Deterministic Finite Automata* merujuk pada sifat utama *automata* tersebut, di mana untuk setiap input yang diterima, hanya ada satu dan tepat satu *state* tujuan yang dapat dituju dari *state* saat ini. DFA memproses urutan simbol input satu demi satu, dan jika setelah seluruh input selesai diproses, mesin ini berakhir di *accepting state*, maka ukuran input tersebut dinyatakan dikenali atau diterima.

Secara formal, sebuah DFA didefinisikan menggunakan notasi 5-tuple, yaitu $A = (Q, \Sigma, \delta, q_0, F)$, dengan keterangan:

- Q : Himpunan *states* yang terbata
- Σ : Himpunan simbol input (*alphabet)* yang terbatas.
- δ : Fungsi transisi yang mengambil argumen berupa *state* saat ini dan sebuah simbol input, lalu mengembalikan tepat satu *state* tujuan.
- q_0 : *State* awal yang merupakan salah satu anggota dari Q
- F : Himpunan *accepting states* yang merupakan himpunan bagian dari Q

II. Perancangan dan Implementasi

A. Rancangan DFA

Secara formal, definisi dari DFA yang dirancang untuk program *lexical analyzer* dijabarkan dalam Tabel 1. DFA ini akan diterapkan pada program *lexer analyzer* agar dapat mengenali *token-token* dalam bahasa pemrograman Arion. DFA ini dirancang untuk bekerja berdampingan dengan struktur data *lookup table* (tabel hash). Ketika membaca karakter alfabet, DFA akan langsung bertransisi ke *state* `ident` dan terus mengakumulasi karakter hingga menemukan karakter pembatas (seperti spasi atau simbol). Setelah state `ident` selesai, program akan mencari *string* yang terkumpul di dalam *keyword lookup table*. Jika ditemukan, *token* akan diklasifikasikan sebagai *keyword;* jika tidak, *token* tetap diklasifikasikan sebagai *identifier*.

Tabel 1. Definisi formal DFA program *lexical analyzer* bahasa pemrograman Arion

Komponen	Anggota

States (Q)	`Start,`
	`CommentCurly,`
	`CommentStar,`
	`CommentStarClose,`
	`Number,`
	`RealBegin,`
	`Real,`
	`CharBegin,`
	`Char,`
	`String,`
	`ident,`
	`sy_minus,`
	`sy_eql,`
	`sy_lss,`
	`sy_gt,`
	`sy_lpar,`
	`sy_colon,`

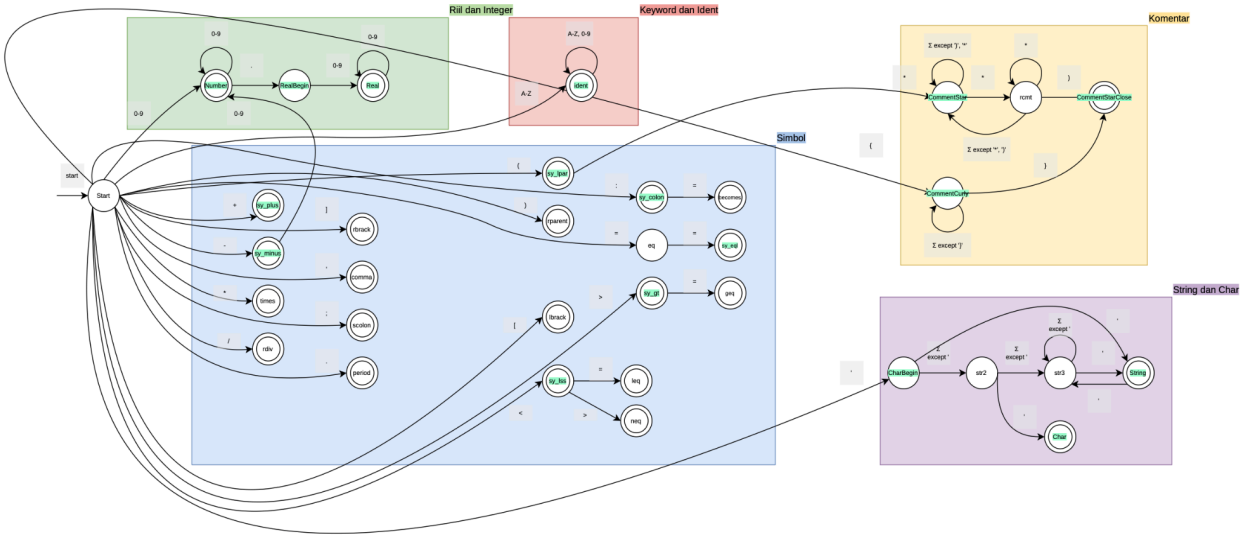
Alphabet (Σ)	Huruf kecil: a-z Huruf besar: A-Z Angka: 0-9 Operator: +, -, *, /, >, <, = Simbol dan tanda baca: :, ;, ,, ., (,), [,], {, }, ' *Whitespace:* spasi (<SPACE>), baris baru (<ENTER>), *carriage return* (<\r>) Karakter lain: Seluruh karakter cetak ASCII lainnya yang tidak termasuk ke dalam himpunan huruf, angka, operator, dan tanda baca di atas.
Transition Function (δ)	**-- Whitespace & Comments --** (Start, `<Whitespace>`) → Start (Start, `{`) → CommentCurly (CommentCurly, `[^}]`) → CommentCurly (CommentCurly, `}`) → Start (sy_lpar, `*`) → CommentStar (CommentStar, `[^*]`) → CommentStar (CommentStar, `*`) → CommentStarClose (CommentStarClose, `*`) → CommentStarClose (CommentStarClose, `)`) → Start (CommentStarClose, `[^*)  ]`) → CommentStar **-- Number & Real --** (Start, `[0-9]`) → Number

	(Number, `[0-9]`) → Number (Number, `.`) → RealBegin (Number, `[^0-9.]`) → Start *(Emit intcon)* (RealBegin, `[0-9]`) → Real (Real, `[0-9]`) → Real (Real, `[^0-9]`) → Start *(Emit realcon)* **-- Char & String --** (Start, `'`) → CharBegin (CharBegin, `[^']`) → Char (Char, `'`) → Start *(Emit charcon)* (Char, `[^']`) → String (String, `[^']`) → String (String, `'`) → Start *(Emit string)* **-- Identifier & Keyword --** (Start, `[a-zA-Z]`) → ident (ident, `[a-zA-Z0-9]`) → ident (ident, `[^a-zA-Z0-9]`) → Start *(Lookup Keyword/Emit ident)* **-- Operators & Symbols --** (Start, `-`) → sy_minus (sy_minus, `[0-9]`) → Number

	(sy_minus, `[^0-9]`) → Start *(Emit minus)* (Start, `=`) → sy_eql (sy_eql, `=`) → Start *(Emit eql)* (Start, `<`) → sy_lss (sy_lss, `>`) → Start *(Emit neq)* (sy_lss, `=`) → Start *(Emit leq)* (sy_lss, `[^>=]`) → Start *(Emit lss)* (Start, `>`) → sy_gt (sy_gt, `=`) → Start *(Emit geq)* (sy_gt, `[^=]`) → Start *(Emit gtr)* (Start, `(`) → sy_lpar (sy_lpar, `[^*]`) → Start *(Emit lparent)* (Start, `:`) → sy_colon (sy_colon, `=`) → Start *(Emit becomes)* (sy_colon, `[^=]`) → Start *(Emit colon)* *(Catatan: Simbol tunggal seperti `*`, `/`, `)`, `[`, `]`, `,`, `;`, `.` langsung dievaluasi di Start dan memicu emit token tanpa pindah state).*
Start state (q_0)	`Start`

Final state (F)	`Number,`
	`Real,`
	`Char,`
	`String,`
	`ident,`
	`sy_minus,`
	`sy_eql,`

	sy_lss, sy_gt, sy_lpar, Sy_colon, CommentCurly, CommentStarClose



Gambar 1. Diagram DFA program lexical analyzer bahasa pemrograman Arion.

B. Implementasi Program

1. Implementasi DFA
   Implementasi program yang digunakan kelompok kami menggunakan variabel `state` dengan struktur switch-case dalam suatu loop. Setiap case dalam kasus ini akan dianggap sebagai sebuah state dalam diagram transisi. Spasi akan dibuang menggunakan fungsi `isWhiteSpace` agar dapat terbuang.
2. Pengenalan Token
   - Token dikenali lewat suatu jalur transisi karakter demi karakter untuk setiap keyword dibandingkan mengecek keseluruhan pada suatu *lookup table*. Jika di tengah jalan karakter tidak sesuai, kode akan secara otomatis mengalihkan status ke ident

melalui fungsi `identBehaviour`.

- Dari segi konstanta, pembedaan integer dan real dilakukan saat pada state Number, ditemukan karakter '.'. DFA langsung berpindah ke state RealBegin untuk melihat konstanta tersebut sebagai realcon.
- Dari segi operator dan simbol, kode sudah menangani apakah tanda '<' bisa berakhir menjadi '<', '<=', atau '<>' tergantung dari karakter selanjutnya. Sama halnya dengan ':' yang bisa menjadi ':' atau ':='.
- Komentar akan ditangani lewat { } atau {* *} terlepas apakah isinya mengandung token yang mungkin invalid.

3. Error Handling
   Error handling sudah ditangani menggunakan fungsi `errorMsg` yang memberikan pesan soal nomor baris mana yang bermasalah.

III. Pengujian

A. Pengujian 1

Input

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Output

```
>> Input .txt filename (from test/milestone-1/input/): tes1
programsy
ident (Hello)
semicolon

varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon

beginsy
ident (a)
becomes
intcon (5)
semicolon

ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon

ident (writeln)
lparent
string ('Result = ')
comma
ident (b)
rparent
semicolon

endsy
period
```

Gambar 3.1 Hasil Pengujian 1

B. Pengujian 2

Input

```
1, 3, 48
3.14, 26.7
'j', 'k', 'h'
'IRK', 'TBFO', 'jkh'
NOT
+
-
*
div
/
MOD
AND
OR
==
<>
>
>=
<
<=
(
)
[
]
,
;
.
:
:=
const
type
var
function
procedure
array
record
program
x, PI, MyInt,
Integer, Real, Char, String
h4g,
identIniSangatPanjang24tetapiTetapValid
begin
if
case
repeat
while
for
end
else
until
```

```
of
do
to
downto
then
{ Ini komentar }, (* In komentar *)
vara
asdasdasf
-67
50
```

Output

```
>> Input .txt filename (from test/milestone-1/input/): tes2
intcon (1)
comma
intcon (3)
comma
intcon (48)
realcon (3.14)
comma
realcon (26.7)
charcon (j)
comma
charcon (k)
comma
charcon (h)
string ('IRK')
comma
string ('TBFO')
comma
string ('jkh')
notsy
plus
minus
times
idiv
rdiv
imod
andsy
orsy
eql
neq
gtr
geq
lss
leq
lparent
rparent
lbrack
rbrack
comma
semicolon

period
colon
becomes
constsy
typesy
varsy
functionsy
proceduresy
```

```
arraysy
recordsy
programsy
ident (x)
comma
ident (PI)
comma
ident (MyInt)
comma
ident (Integer)
comma
ident (Real)
comma
ident (Char)
comma
ident (String)
ident (h4g)
comma
ident (identIniSangatPanjang24tetapiTetapValid)
beginsy
ifsy
casesy
repeatsy
ident (while)
forsy
endsy
elsesy
untilsy
ofsy
arraysy
recordsy
programsy
ident (x)
comma
ident (PI)
comma
ident (MyInt)
comma
ident (Integer)
comma
ident (Real)
comma
ident (Char)
comma
ident (String)
ident (h4g)
comma
ident (identIniSangatPanjang24tetapiTetapValid)
beginsy
```

```
ifsy
casesy
repeatsy
ident (while)
forsy
endsy
elsesy
untilsy
ofsy
comma
ident (PI)
comma
ident (MyInt)
comma
ident (Integer)
comma
ident (Real)
comma
ident (Char)
comma
ident (String)
ident (h4g)
comma
ident (identIniSangatPanjang24tetapiTetapValid)
beginsy
ifsy
casesy
repeatsy
ident (while)
forsy
endsy
elsesy
untilsy
ofsy
ident (String)
ident (h4g)
comma
ident (identIniSangatPanjang24tetapiTetapValid)
beginsy
ifsy
casesy
repeatsy
ident (while)
forsy
endsy
elsesy
untilsy
ofsy
```

```
ifsy
casesy
repeatsy
ident (while)
forsy
endsy
elsesy
untilsy
ofsy
ident (while)
forsy
endsy
elsesy
untilsy
ofsy
elsesy
untilsy
ofsy
ofsy
dosy
tosy
downtosy
thensy
comment
comma
comment
ident (vara)
ident (asdasdasf)
intcon (-67)
intcon (50)
```

Gambar 3.2 Hasil Pengujian 2

C. Pengujian 3

Input

```
1 - 2
1 -2
1- 2
1-2
1 +2
1+ 2
1 +-2
1+- 2
1+-2
1-+2
1-+ 2
1 -+2
```

Output

```
>> Input .txt filename (from test/milestone-1/input/): tes3
intcon (1)
minus
intcon (2)
intcon (1)
intcon (-2)
intcon (1)
minus
intcon (2)
intcon (1)
intcon (-2)
intcon (1)
plus
intcon (2)
intcon (1)
plus
intcon (2)
intcon (1)
plus
intcon (-2)
intcon (1)
plus
minus
intcon (2)
intcon (1)
plus
intcon (-2)
intcon (1)
minus
plus
intcon (2)
intcon (1)
minus
plus
intcon (2)
intcon (1)
minus
plus
intcon (2)
```

Gambar 3.3 Hasil Pengujian 3

D. Pengujian 4

Input

```
{ Menguji operator perbandingan dan logika }
program UjiLogika;
var
  x, y: integer;
  isValid: boolean;
begin
  if (x <> y) AND (x <= 100) OR NOT (y > 50.5) then
    x := x MOD 2;
  else
    y := x div 3;
end.
```

Output

```
>> Input .txt filename (from test/milestone-1/input/): tes4
comment
programsy
ident (UjiLogika)
semicolon

varsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon

ident (isValid)
colon
ident (boolean)
semicolon

beginsy
ifsy
lparent
ident (x)
neq
ident (y)
rparent
andsy
lparent
ident (x)
leq
intcon (100)
rparent
orsy
notsy
lparent
ident (y)
gtr
realcon (50.5)
rparent
thensy
ident (x)
becomes
ident (x)
imod
intcon (2)
semicolon

elsesy
ident (y)
becomes
ident (x)
idiv
intcon (3)
semicolon

endsy
period
```

Gambar 3.4 Hasil Pengujian 4

E. Pengujian 5

Input

```
program Stringggggg;

var
  s1, s2, s3, s4, s5, s6, s7: string;
  c1, c2, c3, c4, c5: char;
  identTest: integer;

begin
  s1 := 'Arion Compiler Milestone 1';
  s2 := '';
  s3 := 'It''s a great day for coding in ''C++''';
  s4 := 'Dia berkata, ''''Halo!''''';
  s5 := 'String dengan simbol: !@#$%^&*()_+';
  s6 := '   String dengan banyak spasi di dalam   ';
  s7 := 'String yang berisi kurawal { ini bukan komentar }';

  c1 := 'A';
  c2 := '7';
  c3 := '+';
  c4 := '''';
  c5 := ' ';

  s1 := 'Cek petik di ujung''';
  s2 := '''' + 'Tengah' + '''';
  s3 := 'Double single quote di awal: ''''Hello';
  s4 := 'Double single quote di tengah: Hel''''lo';
  s5 := 'Double single quote di akhir: Hello''''';

  c1 := 'B';
  if (c1 == 'B') then
  begin
    s1 := 'Kombinasi ''string'' dan ''char''';
  end;

  s6 := 'Line 1' + 'Line 2';
  s7 := 'Tab Test';

  identTest := 100;

  s1 := 'Tes ident di dalam string: var ident begin end';
  s2 := '(* Ini bukan komentar karena di dalam string *)';
  s3 := '{ Ini juga bukan komentar }';

  c2 := 'z';
  c3 := '0';

  s4 := 'Punctuation test: .,;:()[]';
  s5 := 'Sangat '''' Panjang '''' Sekali '''' String '''' Ini';
```

```
  writeln(s1);
  writeln(s2);
  writeln(s3);
  writeln(c1);
  writeln(c4);

  s6 := 'Done';
end.
```

Output

```
>> Input .txt filename (from test/milestone-1/input/): tes5
programsy
ident (Stringggggg)
semicolon

varsy
ident (s1)
comma
ident (s2)
comma
ident (s3)
comma
ident (s4)
comma
ident (s5)
comma
ident (s6)
comma
ident (s7)
colon
ident (string)
semicolon

ident (c1)
comma
ident (c2)
comma
ident (c3)
comma
ident (c4)
comma
ident (c5)
colon
ident (char)
semicolon

ident (identTest)
colon
ident (integer)
semicolon

beginsy
ident (s1)
becomes
string ('Arion Compiler Milestone 1')
semicolon
```

```
ident (s2)
becomes
string ('')
semicolon

ident (s3)
becomes
string ('It's a great day for coding in 'C++'')
semicolon

ident (s4)
becomes
string ('Dia berkata, ''Halo!''')
semicolon

ident (s5)
becomes
string ('String dengan simbol: !@#$%^&*()_+')
semicolon

ident (s6)
becomes
string ('   String dengan banyak spasi di dalam   ')
semicolon

ident (s7)
becomes
string ('String yang berisi kurawal { ini bukan komentar }')
semicolon

ident (c1)
becomes
charcon (A)
semicolon

ident (c2)
becomes
charcon (7)
semicolon

ident (c3)
becomes
charcon (+)
semicolon

ident (c4)
becomes
string ('''')
```

```
semicolon

ident (c5)
becomes
charcon ( )
semicolon

ident (s1)
becomes
string ('Cek petik di ujung'')
semicolon

ident (s2)
becomes
string ('''')
plus
string ('Tengah')
plus
string ('''')
semicolon

ident (s3)
becomes
string ('Double single quote di awal: ''Hello')
semicolon

ident (s4)
becomes
string ('Double single quote di tengah: Hel''lo')
semicolon

ident (s5)
becomes
string ('Double single quote di akhir: Hello''')
semicolon

ident (c1)
becomes
charcon (B)
semicolon

ifsy
lparent
ident (c1)
eql
charcon (B)
```

```
semicolon

ifsy
lparent
ident (c1)
eql
charcon (B)
rparent
thensy
beginsy
ident (s1)
becomes
string ('Kombinasi 'string' dan 'char'')
semicolon

endsy
semicolon

ident (s6)
becomes
string ('Line 1')
plus
string ('Line 2')
semicolon

ident (s7)
becomes
string ('Tab    Test')
semicolon

ident (identTest)
becomes
intcon (100)
semicolon

ident (s1)
becomes
string ('Tes ident di dalam string: var ident begin end')
semicolon

ident (s2)
becomes
string ('(* Ini bukan komentar karena di dalam string *)')
semicolon

ident (s3)
becomes
string ('{ Ini juga bukan komentar }')
```

```
semicolon

ident (c2)
becomes
charcon (z)
semicolon

ident (c3)
becomes
charcon (0)
semicolon

ident (s4)
becomes
string ('Punctuation test: .,;:()[]')
semicolon

ident (s5)
becomes
string ('Sangat '' Panjang '' Sekali '' String '' Ini')
semicolon

ident (writeln)
lparent
ident (s1)
rparent
semicolon

ident (writeln)
lparent
ident (s2)
rparent
semicolon

ident (writeln)
lparent
ident (s3)
rparent
semicolon

ident (writeln)
lparent
ident (c1)
rparent
semicolon
```

```
ident (writeln)
lparent
ident (c1)
rparent
semicolon

ident (writeln)
lparent
ident (c4)
rparent
semicolon

ident (s6)
becomes
string ('Done')
semicolon

endsy
period
```

Gambar 3.5 Hasil Pengujian 5

IV. Kesimpulan dan Saran

A. Kesimpulan

Implementasi program lexer untuk bahasa Arion yang kelompok kami buat telah berhasil memenuhi seluruh spesifikasi tugas besar yang ditetapkan. Program yang dirancang mampu memproses karakter menjadi rangkaian token yang bermakna melalui mekanisme Deterministic Finite Automata (DFA). Seluruh transisi state dalam kode dikembangkan secara manual tanpa bantuan library eksternal untuk mengenali 52 jenis token, mulai dari konstanta numerik, operator, hingga penanganan komentar. Keberhasilan penanganan escape character pada string serta integrasi fungsi pembantu seperti startBehavior memastikan bahwa lexer berjalan stabil, modular, dan mampu memberikan pesan kesalahan yang informatif sesuai dengan tuntutan penilaian.

B. Saran

Untuk pengembangan pada milestone berikutnya, kami berniat hasil token yang sudah dideteksi oleh *lexer* ditampung di dalam sebuah *container* pada memori utama, seperti struktur data *dynamic array* (misalnya `std::vector`) . Setiap token idealnya direpresentasikan sebagai sebuah objek komprehensif yang menyimpan informasi esensial, seperti jenis token, nilai karakter aslinya, dan posisi baris dan kolom terjadinya token. Penyimpanan koleksi token ke dalam *container* ini akan sangat mempermudah tahapan *parsing*.

Referensi

"Compiler Design - Lexical Analysis"
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

"Introduction to Automata Theory, Languages, and Computation 3rd Edition"
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

"Compilers - Principles, Techniques, and Tools (2006)"
https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf

Lampiran

Github : BST-Tubes-IF2224-2026
Workspace DFA : DFA_workspace

Pembagian Tugas :

NIM	Persentase
13524019	25%
13524033	25%
13524043	25%
13524101	25%